
Stop Prompting. Start Looping.

The guy who *built* Claude Code doesn't write prompts anymore. Here's what he does instead — and how you can steal it.

The Quote That Should Make You Feel Behind

Boris Cherny, Head of Claude Code at Anthropic, recently described his own workflow like this:

"I don't prompt Claude anymore. I have loops running that prompt Claude and figure out what to do. My job is to write loops."

Read that again. The person responsible for the most powerful AI coding tool on the planet isn't sitting there typing instructions. He designed a system that does the typing for him.

For the last few years, everyone's been obsessed with prompt engineering. Better prompts, longer prompts, prompts with personas and chain-of-thought and XML tags. And sure, that stuff matters. But here's the problem with prompting: **it keeps you in the room.** You type, you wait, you fix, you type again. You're basically a very expensive middle manager for a robot.

A loop replaces you in that seat. You define what "done" looks like, and the loop finds the work, does it, checks itself, and keeps going until it's actually finished. No babysitting required.

What a Loop Actually Is

Every loop, no matter how fancy, runs the same five-beat cycle:

- 1 Find the work.** The loop figures out what needs doing — open tasks in a list, failing tests, unanswered emails, a folder full of files that need updating.
- 2 Do it.** The agent works one item at a time, the same way you would've prompted it manually. Except you're not there.
- 3 Check itself.** A verification step decides whether the work is actually done and *looking* correct. correct — not just generated and
- 4 Remember.** The loop writes down what it finished so it never repeats work and can pick up where it left off if interrupted.
- 5 Go again.** Repeat until there's nothing left, then stop — or ping you.

That's it. The skill shift isn't about writing bigger prompts. It's about designing this cycle once so the prompting happens without you.

PROMPTING IS TACTICAL

One instruction, one result. You're in the conversation.

LOOP ENGINEERING IS STRATEGIC

You design the system that issues the instructions. You're *above* the conversation.

The Two Commands That Actually Run a Loop

You don't build a loop by typing "do this in a loop" into a chat window. Claude Code has two specific commands built in, and which one you reach for depends on what kind of loop you need.

`/goal`

THE LOOP WITH A FINISH LINE

Use `/goal` when there's a clear definition of done. Type `/goal` followed by what "done" actually looks like, and Claude keeps working turn after turn on its own. The key mechanic: after every single turn, a second AI quietly checks whether you've hit the goal yet. If you haven't, it tells Claude why and Claude keeps going. The moment the goal is truly met, the loop stops on its own.

That self-check after every turn is the whole difference between a real loop and a prompt that runs once and *hopes*.

Example — cleaning up a content backlog:

```
/goal Every draft post in the /drafts folder has a title under 60 characters and a meta description under 160 characters. After each file, print the character counts. Do not touch the body copy. Stop when every file passes, or after 20 files.
```

Example — processing a list of leads:

```
/goal Every contact in leads.csv has been researched: company size, industry, and one recent news item added to new columns. Mark each row "done" as you go. Stop when every row is marked done or after 50 rows.
```

Example — personalizing outreach emails:

```
/goal Every row in prospects.csv has a personalized first line for a cold email, written based on the company's website and LinkedIn page. Add it to a new column called "opener." Mark each row "done" as you go. Stop when every row is complete or after 40 rows.
```

/loop

THE LOOP WITHOUT A FINISH LINE

Use `/loop` when the work isn't "finish a pile" but "keep checking on something." You give it a frequency and a task, and Claude re-runs it on schedule. This is the loop for recurring work: monitoring, inbox triage, daily reports.

Example — morning inbox triage:

```
/loop every morning at 8am, scan my email for anything urgent, summarize it  
in 3 bullet points, and flag anything that needs a same-day response.
```

Example — watching a deployment:

```
/loop 15m check whether my staging site is returning a 200 status. The moment  
it's back up, tell me and stop checking.
```

Example — weekly content audit:

```
/loop every Monday, check our LinkedIn page for posts from the last 7 days,  
pull the engagement numbers, and drop a summary in weekly-social-report.md.
```

Press `Esc` to stop a loop that's waiting for its next run.

The One Thing People Get Wrong

Your goal has to be something Claude can actually grade itself against.

WON'T WORK

"Make it better."
"Clean it up and make it sound professional."

WILL WORK

"Every sentence is under 25 words. Passive voice is eliminated. Reading level is under grade 10."

Fuzzy goals give the loop nothing to check, so it just stops after one pass like a normal prompt. Measurable goals — real numbers, a checklist, a pass/fail condition — are what make the self-check work. **No measurable goal, no real loop.**

Four Pieces That Make Loops More Powerful

The two commands above are the engine. These are the upgrades.

1. Automations

For loops that should run even when no session is open, run Claude headlessly from a scheduler: `claude -p "your loop prompt"`. Scheduled runs are how the loop works while you sleep.

2. Skills

A skill is a markdown file stored in `.claude/skills/` that holds the context you keep re-explaining: project structure, tone guidelines, naming conventions, rules. Write it once and every loop run starts already knowing your setup.

Easiest way to create one: tell Claude **"turn everything I just explained about this project into a skill file."**

3. Connectors

Without connectors, a loop can only work on local files. With MCP connectors, "find the work" can mean "read my inbox," "check my task board," or "pull from my CRM." Add them in Claude Code with `claude mcp add` or through the connectors menu.

4. Sub-agents

The agent doing the work shouldn't be the one grading it — that's like letting a student score their own exam. Claude Code lets you define separate agents in `.claude/agents/` so a builder agent does the task and a reviewer agent independently checks it. This is what makes loops trustworthy enough to run unattended.

A Real Example: The Self-Researching Lead List

Say you've got a spreadsheet with 40 prospects. You want Claude to research each one — company size, what they actually do, a recent news item — and write a personalized cold email opener for each row.

Without a loop, Claude will burn through your list in one shot, make up half the details it can't find, and hand you back a spreadsheet that *looks* complete but is full of confident fabrications. It has no idea it's guessing. There's nothing to tell it otherwise.

With a loop, it works one row at a time. It visits the company's website, checks their LinkedIn, looks for recent news — and only writes the opener once it actually has something real to work with. If it can't find good information, it flags the row instead of inventing something. Each row gets verified before the loop moves to the next.

```
/goal Every row in prospects.csv has a personalized cold email opener based on real, verified information about that company. For each contact: visit their company website, find one specific detail (a product, a recent announcement, a stated goal), and write a 1-2 sentence opener that references it. Add the opener to a new column called "opener" and mark the row "done." If you can't find real information, mark it "needs review" and move on. Stop when every row is marked done or needs review, or after 40 rows.
```

That row-by-row verification — the part where it actually *checks what it found* before writing anything — is the whole point. A single prompt hands you one big output and walks away. A loop earns each line before it moves to the next.

The Loop Charter

For simple jobs, a one-line goal is fine. For anything with multiple steps or a bigger backlog, use this charter. Fill in the brackets and paste the whole thing into Claude Code, or drop the `GOAL` line after `/goal` and use the rest as the playbook.

You are running as a loop, not answering a single prompt. Here is your charter:

GOAL

[What does "done" look like? Be specific. Example: "Every invoice in /invoices has been categorized by vendor and added to expenses.csv with no duplicates."]

HOW TO FIND THE WORK

[Where does the work live? Examples: "Scan the /invoices folder for unprocessed PDFs."
/ "Read TODO.md and treat every unchecked item as a task." / "Check my connected inbox for emails tagged 'needs-reply'."]

HOW TO DO THE WORK

[Step-by-step instructions for handling one item at a time.]

HOW TO CHECK YOUR WORK

[What makes one item "done"? What's the measurable bar? Example: "The row exists in expenses.csv, the amount matches the invoice, and the vendor name is spelled correctly."]

STATE FILE

[Where to track progress. Example: "After completing each item, append the filename and timestamp to loop-progress.md. Never process a file already in that list."]

WHEN TO STOP

[The finish condition plus any guardrails. Example: "Stop when all files are processed, or after 100 items, or if you hit 3 consecutive errors – then tell me what's blocking you."]

The state file is what turns ten separate runs into one continuous process. Without it, every run starts from zero. With it, you can run the same charter on a schedule and the loop always picks up exactly where it left off.

Quick Reference

USE <code>/GOAL</code> WHEN...	USE <code>/LOOP</code> WHEN...
There's a finish line	There's a rhythm, not a finish
You have a backlog to process	You need something checked repeatedly
You want it to stop when it's done	You want it to keep running on schedule

If you only remember one thing: your goal has to be measurable. Numbers, checklists, pass/fail conditions. Vague goals produce one-shot prompts with extra steps. Start with `/goal` . Most of the powerful stuff lives there.

Based on Boris Cherny's public commentary on agentic workflows and Claude Code's loop architecture.